

Linear regression

Hypothesis function: $f_{\mathbf{w},b}(\mathbf{x}) = \mathbf{x}^\top \mathbf{w} + b$,

Objective function: $\frac{1}{m} \sum_{i=1}^m (f_{\mathbf{w},b}(\mathbf{x}_i) - y_i)^2$

Probabilistic perspective: $y = \mathbf{w}^\top \mathbf{x} + e$, where $e \sim \mathcal{N}(0, \sigma^2)$

MLE 推导过程:

- The parameter $\mathbf{w}$ can be learned by [maximum log-likelihood estimation \(MLE\)](#), given the training dataset $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^m$, as follows

$$\mathbf{w}_{MLE} = \arg \max_{\mathbf{w}} \log \mathcal{L}(\mathbf{w}; D), \quad (3)$$

$$\begin{aligned} \log \mathcal{L}(\mathbf{w}; D) &= \log \left(\prod_{i=1}^m p(y_i | \mathbf{x}_i, \mathbf{w}) \right) = \sum_{i=1}^m \log \mathcal{N}(\mathbf{w}^\top \mathbf{x}_i, \sigma^2) \\ &= -m \log(\sigma(2\pi)^{\frac{1}{2}}) - \frac{1}{2\sigma^2} \sum_{i=1}^m (y_i - \mathbf{w}^\top \mathbf{x}_i)^2. \end{aligned} \quad (4)$$

- Removing the constants *w.r.t.* $\mathbf{w}$,

$$\mathbf{w}_{MLE} = \arg \min_{\mathbf{w}} \frac{1}{2} \sum_{i=1}^m (y_i - \mathbf{w}^\top \mathbf{x}_i)^2, \quad (5)$$

Analytical solution:

首先，把 $\mathbf{x}$ 和 $\mathbf{w}$ 变换为如此形式: $\mathbf{x}_i^\top \mathbf{w} = [1, x_1, \dots, x_d]_i$

$$\begin{bmatrix} b \\ w_1 \\ \vdots \\ w_d \end{bmatrix}$$

整合目标函数，我们得到:

- In vector-matrix notation, the squared error loss function can be written compactly using $\mathbf{e} = \mathbf{X}\mathbf{w} - \mathbf{y}$:

$$\begin{aligned} J(\mathbf{w}) &= \mathbf{e}^\top \mathbf{e} \\ &= (\mathbf{X}\mathbf{w} - \mathbf{y})^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) \\ &= (\mathbf{w}^\top \mathbf{X}^\top - \mathbf{y}^\top) (\mathbf{X}\mathbf{w} - \mathbf{y}) \\ &= \mathbf{w}^\top \mathbf{X}^\top \mathbf{X}\mathbf{w} - \mathbf{w}^\top \mathbf{X}^\top \mathbf{y} - \mathbf{y}^\top \mathbf{X}\mathbf{w} + \mathbf{y}^\top \mathbf{y} \\ &= \mathbf{w}^\top \mathbf{X}^\top \mathbf{X}\mathbf{w} - 2\mathbf{y}^\top \mathbf{X}\mathbf{w} + \mathbf{y}^\top \mathbf{y} \end{aligned}$$

对 $\mathbf{w}$ 求导并令其为 0 (因为 convex)，我们得到：

Differentiating $J(\mathbf{w})$ with respect to $\mathbf{w}$ and setting the result to 0 :

$$\begin{aligned}\frac{\partial}{\partial \mathbf{w}} J(\mathbf{w}) &= 0 \\ \frac{\partial}{\partial \mathbf{w}} (\mathbf{w}^\top \mathbf{X}^\top \mathbf{X} \mathbf{w} - 2\mathbf{y}^\top \mathbf{X} \mathbf{w} + \mathbf{y}^\top \mathbf{y}) &= 0 \\ \Rightarrow 2\mathbf{X}^\top \mathbf{X} \mathbf{w} - 2\mathbf{X}^\top \mathbf{y} &= 0 \\ \Rightarrow 2\mathbf{X}^\top \mathbf{X} \mathbf{w} &= 2\mathbf{X}^\top \mathbf{y}\end{aligned}$$

解析解： Learning: $\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$

Prediction: $f_{\mathbf{w},b}(\mathbf{X}_{\text{new}}) = \mathbf{X}_{\text{new}} \hat{\mathbf{w}}$

梯度下降: $\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}}, \quad \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}} = \mathbf{X}^\top (\mathbf{X} \mathbf{w} - \mathbf{y})$

对比：此处 m 是样本个数， d 是样本维度

Closed-form solution $\bar{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$	Gradient descent $\bar{\mathbf{w}} \leftarrow \bar{\mathbf{w}} - \alpha \mathbf{X}^\top (\mathbf{X} \bar{\mathbf{w}} - \mathbf{y})$
No hyper-parameter	Needs to choose α
No need to iterate	Needs many iterations
Complexity $O(d^3 + md^2)$	Complexity $O(T \times md^2)$
Slow if d is very large	Works well when d is large

如果要输出 (预测) 的是向量，我们只要把 $\mathbf{w}$ 扩展成矩阵 ($\mathbf{W}$)，方法是一样的：

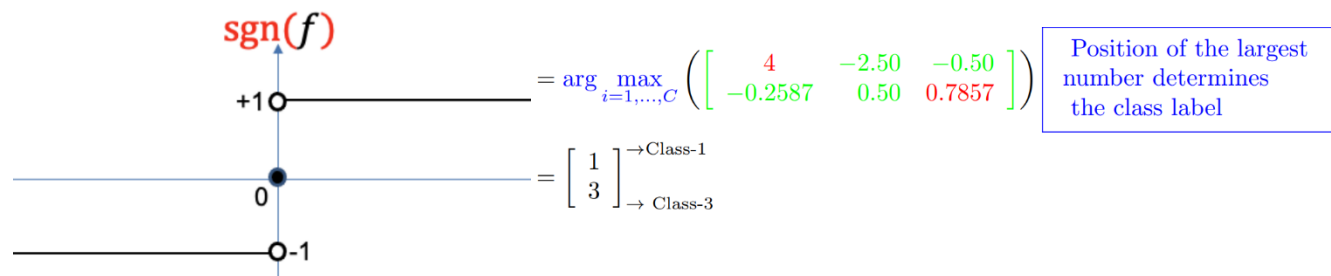
In matrix-matrix notation, the squared error loss function can be written compactly using $\mathbf{E} = \mathbf{XW} - \mathbf{Y}$:

$$\begin{aligned}J(\mathbf{W}) &= \text{trace}(\mathbf{E}^\top \mathbf{E}) \\ &= \text{trace}[(\mathbf{XW} - \mathbf{Y})^\top (\mathbf{XW} - \mathbf{Y})]\end{aligned}$$

If $\mathbf{X}^\top \mathbf{X}$ is invertible, then

Learning: $\hat{\mathbf{W}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{Y}$
 Prediction: $\mathbf{f}_{\mathbf{w},b}(\mathbf{X}_{\text{new}}) = \mathbf{X}_{\text{new}} \hat{\mathbf{W}}$

对于二分类问题，使用 sign 函数；对于多分类问题，取最大值：



Variants:

1. Ridge regression:

Objective function:

$$\sum_{i=1}^m (f_{\mathbf{w},b}(\mathbf{x}_i) - y_i)^2 + \lambda \bar{\mathbf{w}}^\top \bar{\mathbf{w}}, \text{ where } \bar{\mathbf{w}} = \hat{\mathbf{I}}_d \mathbf{w} = [0, w_1, w_2, \dots, w_d]^\top$$

$$\mathbf{w} = \left(\mathbf{X}^\top \mathbf{X} + \lambda \hat{\mathbf{I}}_d \right)^{-1} \mathbf{X}^\top \mathbf{y}$$

Advantage: Guaranteed to be invertible

Probabilistic perspective:

- We can assume that the parameter $\mathbf{w}$ (excluding the bias b) follow a **zero-mean Gaussian prior**

$$p(\mathbf{w}) = \mathcal{N}(\mathbf{w} | \mathbf{0}, \tau^2 \mathbf{I}) \quad (9)$$

- For clarity, **we omit the bias b in the following derivation.**
- Utilizing this prior, we obtain the **maximum a posteriori (MAP)** estimation

$$\mathbf{w}_{MAP} = \arg \max_{\mathbf{w}} \left[\sum_{i=1}^m \log p(y_i | \mathbf{x}_i, \mathbf{w}) + \log p(\mathbf{w}) \right] \quad (10)$$

$$= \arg \max_{\mathbf{w}} \left[\sum_{i=1}^m \log \mathcal{N}(\mathbf{x}_i^\top \mathbf{w}, \sigma^2) + \log \mathcal{N}(\mathbf{w} | \mathbf{0}, \tau^2 \mathbf{I}) \right] \quad (11)$$

$$\equiv \arg \min_{\mathbf{w}} \left[\sum_{i=1}^m (\mathbf{x}_i^\top \mathbf{w} - y_i)^2 + \lambda \|\mathbf{w}\|_2^2 \right]. \quad (12)$$

- The corresponding closed-form solution is given by

$$\mathbf{w}_{MAP} = (\lambda \mathbf{I} + \mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (13)$$

岭回归新添加的项名为惩罚项，旨在解决过拟合问题。

这里是 l2 范数，所以被称作 l2 regularization / weight decay

2. Polynomial regression

举一个二次回归的例子：

- By including terms involving the products of pairs of components of $\mathbf{x}$, we obtain a **quadratic** model:

$$f_{\mathbf{w},b}(\mathbf{x}) = w_0 + \sum_{i=1}^d w_i x_i + \sum_{i=1}^d \sum_{j=1}^d w_{ij} x_i x_j.$$

- In general:

$$f_{\mathbf{w},b}(\mathbf{x}) = w_0 + \sum_{i=1}^d w_i x_i + \sum_{i=1}^d \sum_{j=1}^d w_{ij} x_i x_j + \sum_{i=1}^d \sum_{j=1}^d \sum_{k=1}^d w_{ijk} x_i x_j x_k + \dots$$

注：对 $\mathbf{w}$ 而言，这个模型仍然是线性的，因为这只是线性回归的加长版罢了，

本质上和线性回归相同，只是把 $\mathbf{X}=[1, x_1, x_2, \dots, x_d]$ 变成了 $\mathbf{P}=[1 \quad x_1 \quad x_2 \quad x_1 x_2 \quad x_1^2 \quad x_2^2]$

3. Lasso regression

We can replace the Gaussian prior by a **Laplacian prior**, i.e.,

$$p(\mathbf{w}) = \text{Lap}(\mathbf{w}|\mathbf{0}, \lambda) = \frac{1}{2\lambda} \exp\left(-\frac{|\mathbf{w}|}{\lambda}\right),$$

其余部分和岭回归差不多，把岭回归的二次惩罚项改成一次惩罚项即可

所以 Lasso 回归也叫做 l1 regularization，也可解决过拟合

4. Robust regression

旨在解决噪声问题，减少噪声项对模型的影响，原理也很简单，线性回归的

目标函数是二次项，改成绝对值（l1 范数）即可

注：绝对值不可导，所以不能用梯度下降，只能暴力求解，将

$$\mathbf{w}_{MLE} = \arg \min_{\mathbf{w}} \sum_{i=1}^m |\mathbf{x}_i^\top \mathbf{w} - y_i| \text{ 变为 } \min_{\mathbf{w}, \mathbf{t}} \sum_{i=1}^m t_i$$
$$\text{s.t. } -t_i \leq \mathbf{x}_i^\top \mathbf{w} - y_i \leq t_i, 1 \leq i \leq m.$$

或者使用这种方法，有点类似于 EM:

- We can also utilize the following equation:

$$|a| = \min_{\mu > 0} \frac{1}{2} \left(\frac{a^2}{\mu} + \mu \right) \quad (26)$$

- Then, the above ℓ_1 minimization problem (25) can be reformulated as follows

$$\min_{\mathbf{w}} \min_{\mu_1, \dots, \mu_m > 0} \frac{1}{2} \left(\frac{(\mathbf{x}_i^\top \mathbf{w} - y_i)^2}{\mu_i} + \mu_i \right). \quad (27)$$

- It can be iteratively and alternatively optimized as follows:
 - Given $\mathbf{w}$, $\mu_i = |\mathbf{x}_i^\top \mathbf{w} - y_i|, i = 1, \dots, m$
 - Given μ , $\mathbf{w} = \arg \min_{\mathbf{w}} \sum_{i=1}^m \frac{1}{2} (\mathbf{x}_i^\top \mathbf{w} - y_i)^2$
- It is called **iteratively reweighted least squares** method.

Logistic Regression

旨在解决二分类问题

Hypothesis function: $f_{\mathbf{w},b}(\mathbf{x}) = g(\mathbf{w}^\top \mathbf{x}) \in [0, 1], g(z) = \frac{1}{1 + \exp(-z)}$.

$f_{\mathbf{w},b}(\mathbf{x})$ = estimated probability that $y = 1$ of input $\mathbf{x}$

Objective function: $\frac{1}{2m} \sum_i^m (g(\mathbf{w}^\top \mathbf{x}_i) - y_i)^2$ 注意到目标函数 non-convex

因此改使用 cross-Entropy

Cross-entropy:

$$H(p, q) = - \int_x p(x) \log(q(x)) dx \quad \text{or} \quad - \sum_x p(x) \log(q(x))$$

We set

ground-truth posterior probability : $y(\mathbf{x}) = P(y = 1|\mathbf{x})$,

predicted posterior probability : $f_{\mathbf{w},b}(\mathbf{x}) = P(y = 1|\mathbf{x}; \mathbf{w})$.

Cross-entropy loss:

$$\begin{aligned} \text{cost}(y(\mathbf{x}), f_{\mathbf{w},b}(\mathbf{x})) &= H(y(\mathbf{x}), f_{\mathbf{w},b}(\mathbf{x})) \\ &= - P(y = 1|\mathbf{x}) \cdot \log P(y = 1|\mathbf{x}; \mathbf{w}) - P(y = 0|\mathbf{x}) \cdot \log P(y = 0|\mathbf{x}; \mathbf{w}) \\ &= \begin{cases} -\log(f_{\mathbf{w},b}(\mathbf{x})), & \text{if } y(\mathbf{x}) = 1 \\ -\log(1 - f_{\mathbf{w},b}(\mathbf{x})), & \text{if } y(\mathbf{x}) = 0 \end{cases} \end{aligned}$$

注意到 $f_{\mathbf{w},b}(\mathbf{x})$ 实际上就是 $\mathbf{x}$ 标签为 1 的概率，而 $y(\mathbf{x})$ 只有 100% 和 0% 两种取值

我们可以将目标函数改写为：

$$J(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m \text{cost}(y_i, f_{\mathbf{w},b}(\mathbf{x}_i)),$$
$$\text{cost}(y(\mathbf{x}), f_{\mathbf{w},b}(\mathbf{x})) = \begin{cases} -\log(f_{\mathbf{w},b}(\mathbf{x})), & \text{if } y(\mathbf{x}) = 1 \\ -\log(1 - f_{\mathbf{w},b}(\mathbf{x})), & \text{if } y(\mathbf{x}) = 0 \end{cases}$$
$$J(\mathbf{w}) = -\frac{1}{m} \sum_{i=1}^m [y_i \log(f_{\mathbf{w},b}(\mathbf{x}_i)) + (1 - y_i) \log(1 - f_{\mathbf{w},b}(\mathbf{x}_i))]$$

注意到这是 **convex** 的，可进行梯度下降求解：

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} J(\mathbf{w})$$
$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m [f_{\mathbf{w},b}(\mathbf{x}_i) - y_i] \mathbf{x}_i$$

对于多分类问题，可以采用 **OVR** 策略训练 **c** 个函数，并取最高值

One-vs-all logistic regression:

- Train a binary logistic regression $f_{\mathbf{w}_j, b_j}(\cdot)$ for each class j , by setting all samples of other classes as negative class
- For a new testing sample $\mathbf{x}$, predict its class as $\arg \max_j f_{\mathbf{w}_j, b_j}(\mathbf{x})$.

Pros: Easy to implement

Cons: The training cost is too high, and is difficult to scale to tasks with large number of classes.

Hypothesis function 为：

Softmax function:

$$f_{\mathbf{W}, \mathbf{b}}^{(j)}(\mathbf{x}) = \frac{\exp(\mathbf{w}_j^\top \mathbf{x} + b_j)}{\sum_{c=1}^C \exp(\mathbf{w}_c^\top \mathbf{x} + b_c)} = P(y = j | \mathbf{x}; \mathbf{W}, \mathbf{b})$$

可直观理解为 $\mathbf{x}$ 标签为 j 的概率占总概率的百分比

Objective function 为：

Cost function:

$$J(\mathbf{W}) = -\frac{1}{m} \sum_i \sum_j^C [\mathbb{I}(y_i = j) \log(f_{\mathbf{w}_j, b_j}(\mathbf{x}_i))]$$

where $\mathbb{I}(a) = 1$ if a is true, otherwise $\mathbb{I}(a) = 0$.

可以用梯度下降法求解：

$$\begin{aligned}\frac{\partial J(\mathbf{W})}{\partial \mathbf{w}_j} &= -\frac{1}{m} \sum_i^m \left[\frac{\mathbb{I}(y_i = j)}{f_{\mathbf{w}_j, b_j}(\mathbf{x}_i)} \cdot \frac{\nabla f_{\mathbf{w}_j, b_j}(\mathbf{x}_i)}{\nabla \mathbf{w}_j} \right. \\ &\quad \left. + \sum_{c \neq j}^C \frac{\mathbb{I}(y_i = c)}{f_{\mathbf{w}_c, b_c}(\mathbf{x}_i)} \cdot \frac{\nabla f_{\mathbf{w}_c, b_c}(\mathbf{x}_i)}{\nabla \mathbf{w}_j} \right] \\ \frac{\partial J(\mathbf{W})}{\partial \mathbf{w}_j} &= \frac{1}{m} \sum_i^m (f_{\mathbf{w}_j, b_j}(\mathbf{x}_i) - \mathbb{I}(y_i = j)) \mathbf{x}_i\end{aligned}$$

注：这里的梯度下降是针对 $\mathbf{w}_j$ 的，我们应该平行对所有 $\mathbf{w}_j$ 进行优化，而非顺序

解决 LR 过拟合有两种思路，一是降维，一是加入惩罚项，称为 Regularized LR：

$$\bar{J}(\mathbf{w}) = J(\mathbf{w}) + \frac{\lambda}{2m} \sum_{j=1}^d w_j^2 \quad \text{注意惩罚项实际上就是 } \mathbf{w} \text{ 的 } l_2 \text{ 范数，只不过不包含 } w_0, \text{ 这是因为 } w_0 \text{ 实际上就是偏差 } b, \text{ 所以不会被惩罚}$$

我们依然可以用梯度下降求解，只不过 w_0 和 $\mathbf{w}_j$ 要分开(因为 w_0 无惩罚项)：

$$\begin{aligned}w_0 &\leftarrow w_0 - \frac{\alpha}{m} \sum_{i=1}^m (f_{\mathbf{w}, b}(\mathbf{x}_i) - y_i) \cdot \mathbf{x}_i(0), \text{ where } \mathbf{x}_i(0) = 1, \forall i \\ w_j &\leftarrow w_j - \frac{\alpha}{m} \left[\sum_{i=1}^m (f_{\mathbf{w}, b}(\mathbf{x}_i) - y_i) \cdot \mathbf{x}_i(j) + \lambda \cdot w_j \right],\end{aligned}$$

Probabilistic perspective:

- Behind **logistic regression for binary classification**, we assume that both the feature $\mathbf{x}$ and the label y are random variables, as follows

$$\begin{aligned}\mu(\mathbf{x}; \mathbf{w}) &= \text{Sigmoid}(\mathbf{w}^\top \mathbf{x}), \\ y(\mathbf{x}; \mathbf{w}) &\sim \text{Bernoulli}(\mu(\mathbf{x}; \mathbf{w})).\end{aligned}$$

- Then, we have

$$P(y|\mathbf{x}; \mathbf{w}) = \begin{cases} \mu & \text{if } y = 1, \\ 1 - \mu & \text{if } y = 0. \end{cases}$$

- The **log-likelihood** function of $P(y|\mathbf{x}; \mathbf{w})$ is formulated as

$$\mathcal{L}(\mathbf{w}) = y \log(\mu) + (1 - y) \log(1 - \mu).$$

- Thus, we obtain

$$\max_{\mathbf{w}} \mathcal{L}(\mathbf{w}) \equiv \min_{\mathbf{w}} J(\mathbf{w}).$$

	Linear regression	Logistic regression
Task	regression	classification
Hypothesis $f_{\mathbf{w},b}(\mathbf{x})$	$\mathbf{w}^\top \mathbf{x} + b \in (-\infty, \infty)$	$g(\mathbf{w}^\top \mathbf{x} + b) \in [0, 1]$
Objective $J(\mathbf{w})$	$\frac{1}{2m} \sum_i^m (y_i - \mathbf{w}^\top \mathbf{x}_i)^2$	$-\frac{1}{m} \sum_{i=1}^m [y_i \log(f_{\mathbf{w},b}(\mathbf{x}_i)) + (1 - y_i) \log(1 - f_{\mathbf{w},b}(\mathbf{x}_i))]$
Solution	closed-form or gradient descent	gradient descent

这两种模型从概率角度和优化角度出发会得到相同的目标函数

SVM

Hypothesis function: $y = \text{Sgn}(f_{\mathbf{w}}(\mathbf{x})) = \text{Sgn}(\mathbf{w}^\top \mathbf{x})$

目标函数推导过程：最大化边界

Lemma 1: $\mathbf{x}$ has distance $\frac{|f_{\mathbf{w}}(\mathbf{x})|}{\|\mathbf{w}\|}$ to the the hyperplane $f_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} = 0$

这条很容易理解，代入进去就是 $x \cos \theta$

Claim 1: $\mathbf{w}$ is orthogonal to the hyperplane $f_{\mathbf{w},b}(x) = \mathbf{w}^\top \mathbf{x} + b = 0$

这也容易理解，法向量就是这么定义的

Claim 2: $\mathbf{0}$ has distance $\frac{-b}{\|\mathbf{w}\|}$ to the hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$

显而易见

Lemma 2: $\mathbf{x}$ has distance $\frac{|f_{\mathbf{w},b}(\mathbf{x})|}{\|\mathbf{w}\|}$ to the hyperplane $f_{\mathbf{w},b}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b = 0$

结合 lemma1 和 claim2 可证

所以我们的目标函数为: $\gamma = \min_i \frac{y_i f_{\mathbf{w},b}(\mathbf{x}_i)}{\|\mathbf{w}\|}$ y_i 项是为了让负距离取正

我们要最大化目标函数，而注意到同除一个常数，目标函数不变

$$\frac{y_i (c\mathbf{w}^\top \mathbf{x}_i + cb)}{\|c\mathbf{w}\|} = \frac{y_i (\mathbf{w}^\top \mathbf{x}_i + b)}{\|\mathbf{w}\|}$$

那么不妨让分子为 1，即 $y_{i^*} (\mathbf{w}^\top \mathbf{x}_{i^*} + b) = 1$ ，则目标函数为 $\frac{1}{\|\mathbf{w}\|}$

最大化目标函数等价于最小化 $\mathbf{w}$ 的范数，即

求解：

首先回忆拉格朗日方程和 KKT 条件

- Given a general minimization problem

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^n} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & h_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, m \\ & \ell_j(\mathbf{x}) = 0, \quad j = 1, \dots, r \end{aligned}$$

Note that here $\mathbf{x}$ denotes the argument we aim to optimize, rather than a data point.

- The **Lagrangian function**:

$$L(\mathbf{x}, \mathbf{u}, \mathbf{v}) = f(\mathbf{x}) + \sum_{i=1}^m u_i h_i(\mathbf{x}) + \sum_{j=1}^r v_j \ell_j(\mathbf{x})$$

- The **Lagrange dual function**:

$$g(\mathbf{u}, \mathbf{v}) = \min_{\mathbf{x} \in \mathbb{R}^n} L(\mathbf{x}, \mathbf{u}, \mathbf{v})$$

- The **dual problem**:

$$\begin{aligned} \max_{\mathbf{u} \in \mathbb{R}^m, \mathbf{v} \in \mathbb{R}^r} \quad & g(\mathbf{u}, \mathbf{v}) \\ \text{subject to} \quad & \mathbf{u} \geq 0 \end{aligned}$$

Given general problem

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^n} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & h_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, m \\ & \ell_j(\mathbf{x}) = 0, \quad j = 1, \dots, r \end{aligned}$$

The **Karush-Kuhn-Tucker conditions** or **KKT conditions** are:

- $0 \in \partial f(\mathbf{x}) + \sum_{i=1}^m u_i \partial h_i(\mathbf{x}) + \sum_{j=1}^r v_j \partial \ell_j(\mathbf{x})$ (stationarity)
- $u_i \cdot h_i(\mathbf{x}) = 0$ for all i (complementary slackness)
- $h_i(\mathbf{x}) \leq 0, \ell_j(\mathbf{x}) = 0$ for all i, j (primal feasibility)
- $u_i \geq 0$ for all i (dual feasibility)

这里我们的做法就是，构造拉格朗日函数，再把拉格朗日函数用 KKT 求解（也

就是完成 dual function 步）。注意我们的变量永远是 $\mathbf{w}$ 和 $\mathbf{b}$ 。接着，我们用 KKT

得到的等式，将 $\mathbf{x}$ 尽量表达成 $\mathbf{u}$ 和 $\mathbf{v}$ 。最后，我们对 dual problem 进行求解。

$$\begin{aligned} \max_{\alpha} \quad & \sum_i^m \alpha_i - \frac{1}{2} \sum_{i,j}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j \\ \text{s.t.} \quad & \sum_i^m \alpha_i y_i = 0, \quad \alpha_i \geq 0, \quad \forall i \end{aligned}$$

最终结果: $\mathbf{w} = \sum_i^m \alpha_i y_i \mathbf{x}_i$

Solution interpretation:

- The primal solution $\mathbf{w}$ and the dual solution α should also satisfy other KKT conditions
 - Feasibility: $\alpha_i \geq 0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b) \leq 0, \forall i$
 - Complementary slackness: $\alpha_i(1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)) = 0, \forall i$
- When comparing above conditions together, we have that for $\mathbf{x}_i, \forall i$,
 - If it satisfies $1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b) < 0$, then $\alpha_i = 0$;
 - If it satisfies $1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 0$, then $\alpha_i \geq 0$.
- If $\alpha_i = 0$, then it means that $\mathbf{x}_i$ doesn't contribute to $\mathbf{w}$, i.e., the SVM classifier
- The data points with $\alpha_i > 0$ construct the classifier, and they are called **support vectors**, which locate at the hyperplanes $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$. And, we define the **support set as $\mathcal{S} = \{i | \alpha_i > 0\}$** . This is why we call it **support vector machine**.

注意到 $y_j(\mathbf{w}^\top \mathbf{x}_j + b) = 1$, 直接求解 b 即可

$$\sum_i^m \alpha_i y_i \mathbf{x}_i^\top \mathbf{x}_j + b = y_j, \forall j \in \mathcal{S}$$

$$\Rightarrow b = \frac{1}{|\mathcal{S}|} \sum_{j \in \mathcal{S}} (y_j - \sum_i^m \alpha_i y_i \mathbf{x}_i^\top \mathbf{x}_j)$$

$$\mathcal{S} = \{i | \alpha_i > 0\}$$

软间隔 SVM (SVM with slack variable)

To handle such data, we introduce **slack variable $\xi_i \geq 0$**

We allow some errors for training data, i.e., $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \forall i$, rather than $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \forall i$

But we hope that such errors $\xi_i, \forall i$ are small

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i^m \xi_i$$

Objective function:

$$s.t. 1 - \xi_i - y_i(\mathbf{w}^\top \mathbf{x}_i + b) \leq 0, -\xi_i \leq 0, \forall i$$

同样, 构造拉格朗日方程并求解。直接给出结果:

$$\mathcal{L}(\mathbf{w}, b, \xi, \alpha, \mu) = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i^m \xi_i + \sum_i^m [\alpha_i (1 - \xi_i - y_i(\mathbf{w}^\top \mathbf{x}_i + b)) + \mu_i (-\xi_i)]$$

- Replacing all stationary conditions into Lagrange function to eliminate primal variables, we have

$$\begin{aligned}\mathcal{L}(\alpha, \mu) &= \frac{1}{2} \|\mathbf{w}\|^2 + \sum_i^m [\alpha_i (1 - y_i (\mathbf{w}^\top \mathbf{x}_i + b))] + \sum_i^m (C - \alpha_i - \mu_i) \xi_i \quad (32) \\ &= \sum_i^m \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j.\end{aligned}$$

Utilizing $\alpha_i = C - \mu_i$, we obtain a simpler dual problem:

$$\begin{aligned}\max_{\alpha} \quad & \sum_i^m \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j, \\ \text{s.t.} \quad & \sum_i^m \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C, \quad \forall i\end{aligned}$$

同理，b 可直接求解：

$$\begin{aligned}\sum_i^m \alpha_i y_i \mathbf{x}_i^\top \mathbf{x}_j + b &= y_j, \quad \forall j \in \mathcal{M} \\ \Rightarrow b &= \frac{1}{|\mathcal{M}|} \sum_{j \in \mathcal{M}} (y_j - \sum_i^m \alpha_i y_i \mathbf{x}_i^\top \mathbf{x}_j) \quad \mathcal{M} = \{i | 0 < \alpha_i < C\}\end{aligned}$$

解释：

Solution interpretation

- The solution α_i has three cases: $\alpha_i = 0, 0 < \alpha_i < C, \alpha_i = C$
- $\alpha_i = 0$: the corresponding data are correctly classified and doesn't contribute to the classifier, locating **outside** of the margin
- $0 < \alpha_i < C$: in this case, $\mu_i > 0$ due to $\alpha_i = C - \mu_i$; Since $\mu_i \xi_i = 0$, then we have $\xi_i = 0$. The corresponding data are correctly classified and contributes to the classifier, locating **on** the margin
- $\alpha_i = C$: in this case, $\mu_i = 0$; then we have $\xi_i > 0$. The corresponding data contributes to the classifier, locating **inside** the margin
 - If $\xi_i \leq 1$, then the data is still correctly classified, not crossing decision boundary
 - If $\xi_i > 1$, then the data is incorrectly classified, crossing decision boundary

注：事实上，对于这两种 SVM 的目标函数，不用拉格朗日函数，直接使用 KKT

条件也是可以的。构造是为了让解的结构更清晰

SVM with kernel

类比 polynomial regression, 我们也可以让 SVM 不使用线性模型

We firstly define the following kernel

$$k(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$$

The prediction of new data $\mathbf{x}$ becomes

$$\mathbf{w}^\top \mathbf{x} + b = \sum_i^m \alpha_i y_i k(\mathbf{x}_i, \mathbf{x}) + b$$

Widely used kernels:

Polynomial kernel: $k(\mathbf{x}, \mathbf{x}_i) = \left(1 + \frac{\mathbf{x}^\top \mathbf{x}_i}{\sigma^2}\right)^p, p > 0$

Radial Basis Function (RBF) kernel: $k(\mathbf{x}, \mathbf{x}_i) = \exp\left\{-\frac{\|\mathbf{x} - \mathbf{x}_i\|^2}{2\sigma^2}\right\}$

Sigmoidal kernel: $k(\mathbf{x}, \mathbf{x}_i) = \frac{1}{1 + \exp^{-\frac{\mathbf{x}^\top \mathbf{x}_i + b}{\sigma^2}}}$

对于多分类问题, 同样使用 OVR 策略

注: 软间隔 SVM 是一类有超参数的模型, 因为要选择 C。C 越大, 边界越松

Decision Tree

Hypothesis function: None

Stopping criterion: 1. A node is pure

2. Maximum depth is reached

3. Performance metric is achieved

Attribute selection:

由于每次切分只考虑一个维度的属性, 我们需要挑最能纯化节点的属性

Classification tree:

Entropy: Entropy for multi-class node S : $\text{Info}(S) = -\sum_{i=1}^K p_i \log_2 p_i$.

Entropy 越高，纯度越低（种类就越杂）。Entropy=Info

Suppose we use an attribute $A \in \{a_1, a_2, \dots, a_V\}$ to split (divide) S into V subsets: $D_1, D_2, \dots, D_V$.

The information of the partition by A is the conditional entropy* (the entropy of S given the condition A)

$$\text{Info}(S|A) = \sum_{v=1}^V \frac{|D_v|}{|S|} \times \text{Info}(D_v)$$

Information gained by branching on attribute A

$$\text{Gain}(A) = \text{Info}(S) - \text{Info}(S|A)$$

比如西瓜的直径有{5, 6, 7}三个取值，那么用直径这个属性进行切分的信息增益

就是 S 的总信息减去分割出的子集的信息乘以他们的权重（权重=数量占比）

其它衡量指标：Gini Index

Gini index: $\text{gini}(S) = 1 - \sum_{i=1}^K p_i^2$

Suppose we use an attribute A to split S into V subsets $D_1, D_2, \dots, D_V$ then the expected Gini index given A is defined as

$$\text{gini}(S|A) = \sum_{v=1}^V \frac{|D_v|}{|S|} \text{gini}(D_v)$$

Reduction in impurity (Gini index):

$$\Delta \text{gini}(A) = \text{gini}(S) - \text{gini}(S|A)$$

Info (entropy) v.s. Gini index

- Gini index has a simpler computation.
- Gini index is more interpretable.
- Info is more effective when the classes are imbalanced.
- Info is less sensitive to noise.

Regression tree:

这里，我们使用 SSE（误差平方的总和，sum of squared errors）作为衡量指标

In regression, the goodness of a split is measured by the **mean square error** (MSE) or the **sum of squared errors** (SSE) from the estimated value.

The **prediction** for leaf c is $\bar{y}_c = \frac{\sum_{i \in \text{leaf}_c} y_i}{N_c}$.

Within each leave c , the MSE can be computed as $e_c = \frac{\sum_{i \in c} (y_i - \bar{y}_c)^2}{N_c}$

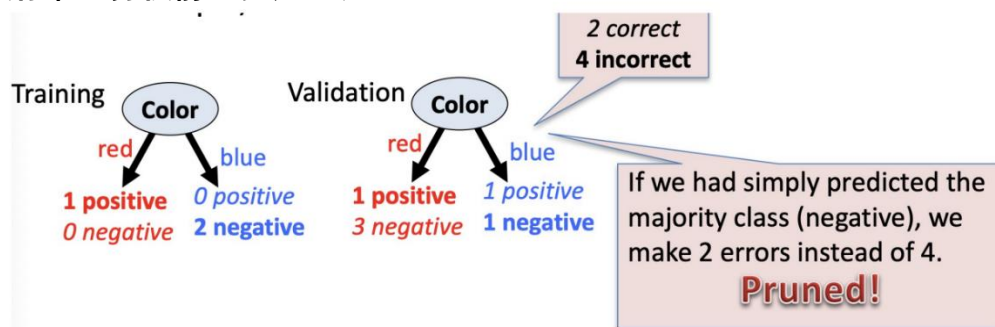
The total MSE is $\mathcal{S} = \sum_{c \in \text{leaves}(\text{Tree})} e_c$.

The total SSE is $\mathcal{S} = \sum_{c \in \text{leaves}(\text{Tree})} \sum_{i \in c} (y_i - \bar{y}_c)^2$. (preferred)

Stopping criterion: SSE 的减少量低于某个边界值

剪枝: 解决过拟合问题

剪枝标准: 剪枝前后验证集误差



总结: 决策树的主要问题是过拟合和对连续值的显著劣势

Bagging:

采取 **data-level randomness**: 用重采样的方法，允许同一个样本被收集多次，并以此制造多个决策树，取众数

Random Forest:

和 bagging 相同的取样策略，同时引入 **attribute-level randomness**: 只考虑某一部分特征，而不是所有特征

所以有 Prediction error:

Random forest < Bagging

< single trees

Each time a split is to be performed, the search for the split attribute is limited to a random subset of m of the N attributes

- For regression trees:
 $m = \frac{N}{3}$
- For classification trees:
 $m = \sqrt{N}$

*根据本人实验，增加树的数量不会显著影响 RF 的误差，但会显著降低方差

Aspect	Random Forest	Decision Tree
Nature	Ensemble of multiple decision trees	Single decision tree
Bias-Variance Trade-off	Lower variance, reduced overfitting	Higher variance, prone to overfitting
Predictive Accuracy	Generally higher due to ensemble	Prone to overfitting, may vary
Robustness	More robust to outliers and noise	Sensitive to outliers and noise
Training Time	Slower due to multiple tree construction	Faster as it builds a single tree
Interpretability	Less interpretable due to ensemble	More interpretable as a single tree
Feature Importance	Provides feature importance scores	Provides feature importance, but less reliable
Usage	Suitable for complex tasks, high-dimensional data	Simple tasks, easy interpretation

Neural Network

Perceptron

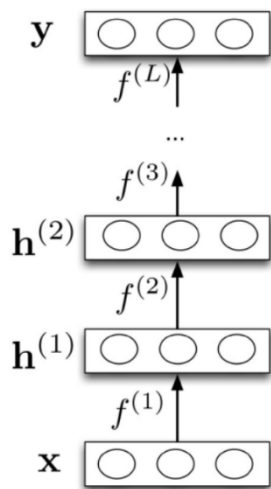
Hypothesis function: Depending on layers

Objective function: $J(\mathbf{w}) = \frac{1}{2}(y - \hat{y})^2$

Multi-layer feedforward neural networks

1. 单向性，能且只能从后方的神经元传播到前方的神经元
2. 全连接，前后两层神经元是 fully connected 的，即后层的每个神经元都会与前层的所有神经元连接
3. 同层无连接，即同层的神经元之间没有连接

易知，我们最后得到的神经网络应该长这样



$$\begin{aligned} \mathbf{h}^{(1)} &= f^{(1)}(\mathbf{x}) \\ \mathbf{h}^{(2)} &= f^{(2)}(\mathbf{h}^{(1)}) \\ &\vdots \\ \mathbf{y} &= f^{(L)}(\mathbf{h}^{(L-1)}) \end{aligned}$$

$$\mathbf{y} = f^{(L)} \circ \dots \circ f^{(1)}(\mathbf{x})$$

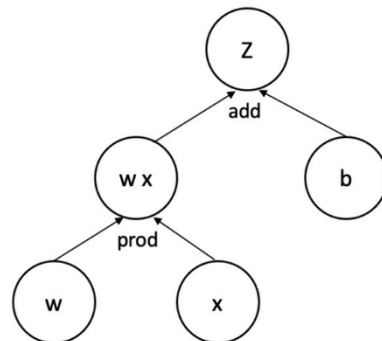
Backpropagation:

利用链式法则，训练神经网络 $\frac{df(h(w))}{dw} = \frac{df}{dh} \cdot \frac{dh}{dw}$.

——computational graph (CG)

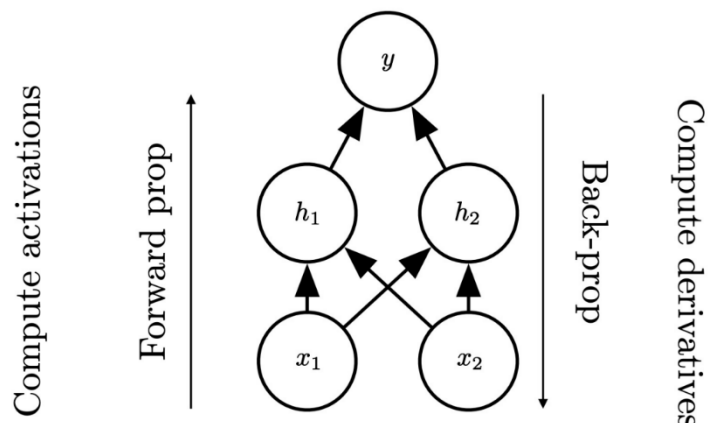
可以把函数写成这种形式，

For example, $z = wx + b$ can be represented by the following CG



Compute loss

这样观察导数之间的关系
会方便很多
当进行预测时，我们的顺序是从后向前；当进行训练时，我们的顺序时从前向后计算导数，然后用梯度下降求解



Computational cost:

Basics: $\mathbf{A}^{a \times b} \cdot \mathbf{B}^{b \times c}$ takes $O(a \cdot b \cdot c)$; $\frac{d}{d} \frac{\mathbf{a}^{d_1 \times 1}}{\mathbf{b}^{d_2 \times 1}}$ takes $O(d_1 \cdot d_2)$

Forward pass (calculating activation of each layer):

$$y = g_1(\mathbf{w}^\top \mathbf{h} + b), \mathbf{h} = g_2(\mathbf{W}\mathbf{x} + \mathbf{c})$$

where $\mathbf{W} \in \mathbb{R}^{m \times d}$, $\mathbf{x} \in \mathbb{R}^{d \times 1}$, $\mathbf{h}, \mathbf{w} \in \mathbb{R}^{m \times 1}$. The cost is

$$O_F = O(md + m).$$

Backward pass (calculating parameter derivative of each layer): $\frac{dL}{d\mathbf{w}} = \frac{dL}{dy} \cdot \frac{dy}{d\mathbf{w}}$, $\frac{dL}{d\mathbf{W}} = \frac{dL}{dy} \cdot \frac{dy}{d\mathbf{h}} \cdot \frac{d\mathbf{h}}{d\mathbf{W}}$.
The computational cost is:

$$O_B = O_{\mathbf{w}}(1 + m + m) + O_{\mathbf{W}}(m + m^2d + m + m^2d) = O(2(m^2d + 2m))$$

解释: $1=dL/dy$; $m=dy/dw$; $m=$ 二者乘积; $m=dy/dh$; $m^2d=dh/dW$; 最后两项分别是 1、2 的乘积和 $1 \cdot 2 \cdot 3$ 的乘积

事实上, 这种神经网络只是把 LR 拆开了而已。真正使其独立的是 activate function, 也就是最后的激活函数。我们可以采用非线性的函数作为激活函数

CNN

不同于普通的 NN, CNN 并不具有全连接的性质 (Sparse connection), 且具有 shared parameter (Parameters at different spatial locations are shared)

CNN 一般用于图片识别。有几个重要的组成部分 (layer)

以下均假设原图片为 $32 \times 32 \times 3$

1. Convolution Layer

我们用一个含有相同厚度的 filter 进行卷积 (filter 的厚度永远与原图片相同)

Stride: 每次移动的步长

假设图片长度为 N , filter 长度为 F , 步长为 S

我们的 Output Size = $(N - F) / S + 1$

在实际中, 我们往往会给图片加一层“边框”,

即 zero-padding。显然, 我们相当于在原图

上加了两倍的 P 的长度。所以输出大小为:

$$(N + 2P - F) / \text{stride} + 1$$

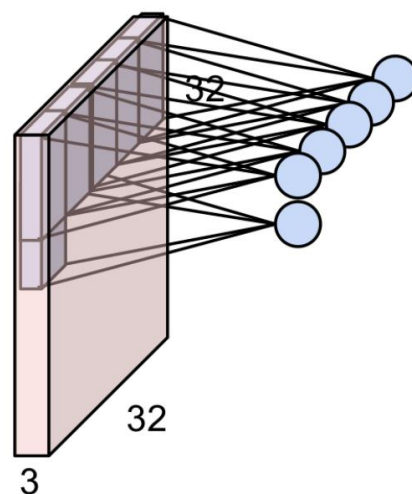
我们不会只用一个 filter 进行卷积。大部分

情况下, 我们会用多个相同的 filter 进行操作,

最后把他们叠在一起。例如, 有 10 个

$F=5, S=1$ 的 filter, 最后的输出结果就是:

$$32 \times 32 \times 10$$



0	0	0	0	0	0				
0									
0									
0									
0									

那么在这个过程中, 我们需要多少个参数呢? 答案是 $(F^2 \times \text{depth} + 1)$

*Number of filters。本例中, 参数是 $5 \times 5 \times 3 + 1 = 76 \times 10 = 760$

Let's assume input is $W_1 \times H_1 \times C$ Conv layer needs 4 hyperparameters:

- Number of filters K
- The filter size F
- The stride S
- The zero padding P

This will produce an output of $W_2 \times H_2 \times K$ where:

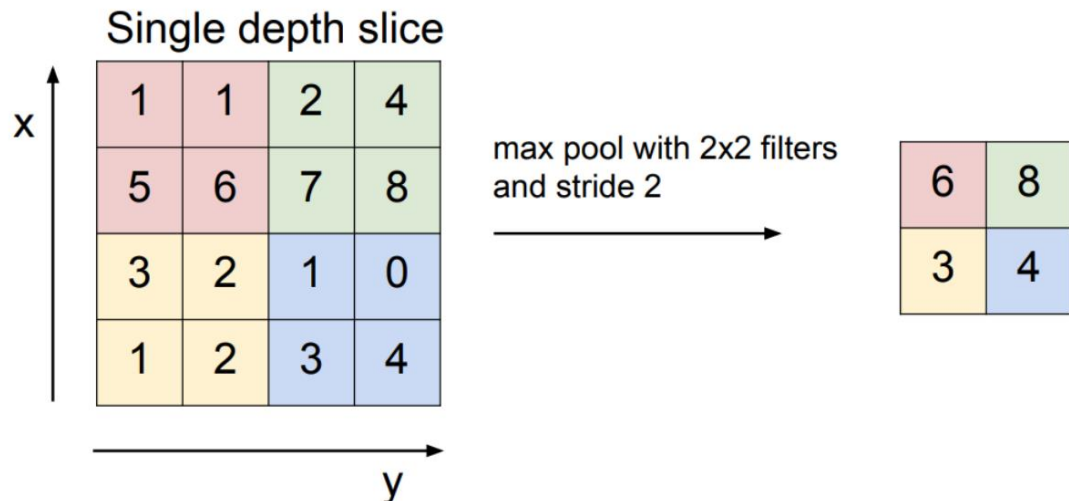
$$- W_2 = (W_1 - F + 2P) / S + 1$$

$$- H_2 = (H_1 - F + 2P) / S + 1$$

Number of parameters: $F^2 C K$ and K biases

2. Pooling layer

makes the representations smaller and more manageable



可以看出，pooling 是一个取样的过程，不包含任何 parameter

3. Fully connected layer (FC layer)

这个过程会把前一步的图拉长，比如前一步的维度是 $8 \times 8 \times 20$ ，这一步便会直接被拉长为 1280；同时，每个 FC layer 还会有自己的输出，比如 FC10，输出一个 1×10 的向量。所以他的形状是 1280×10 ，偏差是 10.总参数是 $12800 + 10 = 12810$

总结一下：Conv layer 的计算方式是 $(N \times \text{filter 的体积} + 1) \times \text{Number of filter}$

FCM (M 代表输出维度，如 FC10 中 $M=10$) = $(\text{前一层的体积} + 1) \times M$

RNN (了解概念即可)

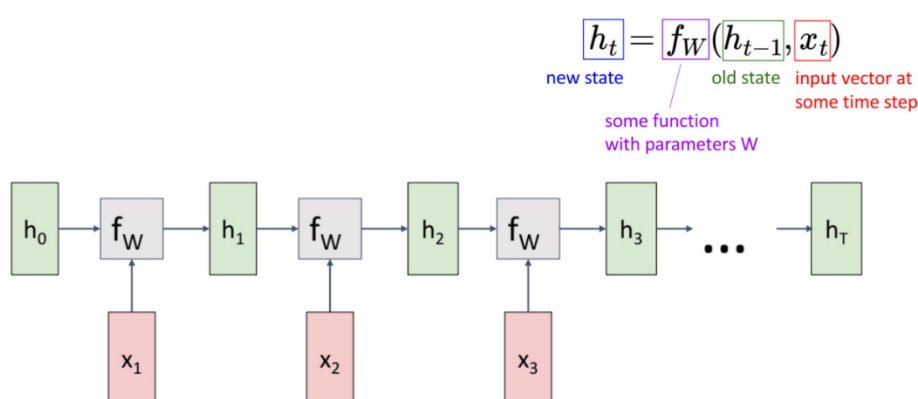
用于处理有时间序列的问题 (如 NLP)

- RNN model

$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state some function with parameters W old state input vector at some time step

RNN computational graph



RNN 与前两种神经网络显著的区别是层之间有连接

$$\text{Model: } h_t = f_W(h_{t-1}, x_t), \quad \hat{y}_t = g_{W'}(h_t)$$

$$\text{Cost/error function: } E(\theta) = \frac{1}{T} \sum_{t=1}^T \mathcal{L}(y_t, \hat{y}_t), \text{ where } \theta = \{W, W'\}$$

Optimization: backpropagation and gradient descent

RNN 的问题，一个是梯度爆炸，计算成本高；一个是梯度消失，长期记忆差

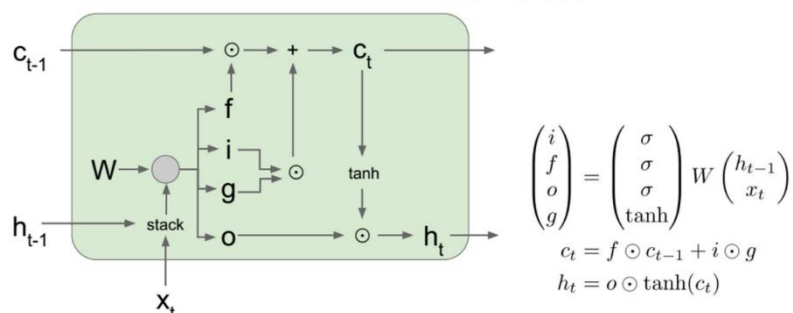
LSTM (Long Short-term Memory)

旨在解决梯度消失问题

Long Short Term Memory (LSTM)
[Hochreiter et al., 1997]

i: Input gate, whether to write to cell
f: Forget gate, Whether to erase cell
o: Output gate, How much to reveal cell
g: Gate gate (?), How much to write to cell

不必细究



Transformer

用于翻译、问答系统，有效解决长期记忆问题，通过自注意机制获取某个词和其他所有词的关系，并通过位置编码生成序列

- Let $\mathbf{X} \in \mathbb{R}^{n \times d}$ (or $\mathbf{H}$ in a hidden layer) be the embedding matrix, where n is the number of samples (e.g., words). Let $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d' \times d}$ be three parameter matrices.
- Let $\mathbf{Q} = \mathbf{XW}_Q, \mathbf{K} = \mathbf{XW}_K, \mathbf{V} = \mathbf{XW}_V$. They are often called ‘query’, ‘key’, and ‘value’ respectively.
- The (self-) [attention mechanism](#) is defined as

$$\mathbf{Z} = \text{attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{QK}^\top}{\sqrt{d'}} \right) \mathbf{V}$$

- $\mathbf{A} := \text{softmax} \left(\frac{\mathbf{QK}^\top}{\sqrt{d'}} \right) \in \mathbb{R}^{n \times n}$ is called [attention map](#).
- When $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ are computed using different features, the mechanism is called cross-attention.

Overfitting, bias-variance tradeoff

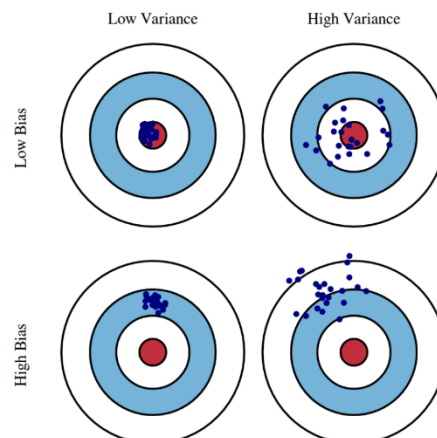
1. Underfitting

欠拟合通常是由于模型过于简单或特征与标签的关联度较差，解决方法是使用更复杂的模型或进行特征工程提取最有价值的特征（如 PCA）

2. Overfitting

过拟合的原因通常是模型过于复杂，训练轮数过多或训练集样本数太少

3. Bias-variance tradeoff



一般来说，训练集误差会随着模型复杂度的提高而降低，最终到 0。当模型复杂度较低的时候，测试集偏差较大，方差较小；当模型复杂度高的时候，测试集偏差较小，方差较大。测试集误差会在某个时刻到达下界

背后的数学逻辑：

The relationship between the input features $\mathbf{x}$ and the output y is

$$y = t(\mathbf{x}) + e, \quad e \sim \mathcal{N}(0, \sigma^2), \quad (1)$$

$$p(y|\mathbf{x}) = \mathcal{N}(t(\mathbf{x}), \sigma^2 \mathbf{I}), \quad (2)$$

where $t(\mathbf{x})$ can be seen as the unknown target function and the mean of $p(y|\mathbf{x})$.

如果真实函数与标签的误差服从正态分布，那么给定 $\mathbf{x}$ ，标签为 y 的概率也是正态分布，这是可以理解的。

- The **expected test error** can be decomposed as follows

$$\begin{aligned} E_{(\mathbf{x}, y), D} [(h_D(\mathbf{x}) - y)^2] &= E_{(\mathbf{x}, y), D} [(h_D(\mathbf{x}) - \bar{h}(\mathbf{x})) + (\bar{h}(\mathbf{x}) - y)]^2 \\ &= E_{(\mathbf{x}, y), D} [(h_D(\mathbf{x}) - \bar{h}(\mathbf{x}))^2] + 2E_{(\mathbf{x}, y), D} [(h_D(\mathbf{x}) - \bar{h}(\mathbf{x}))(\bar{h}(\mathbf{x}) - y)] \\ &\quad + E_{(\mathbf{x}, y), D} [(\bar{h}(\mathbf{x}) - y)^2] \end{aligned}$$

- We have

$$\begin{aligned} &E_{(\mathbf{x}, y), D} [(h_D(\mathbf{x}) - \bar{h}(\mathbf{x})) \cdot (\bar{h}(\mathbf{x}) - y)] \\ &= E_{(\mathbf{x}, y)} [E_D[h_D(\mathbf{x}) - \bar{h}(\mathbf{x})] \cdot (\bar{h}(\mathbf{x}) - y)] \\ &= E_{(\mathbf{x}, y)} [(E_D[h_D(\mathbf{x})] - \bar{h}(\mathbf{x})) \cdot (\bar{h}(\mathbf{x}) - y)] = 0 \end{aligned}$$

- Replace the above equation to the expected test error, then we have

$$E_{(\mathbf{x}, y), D} [(h_D(\mathbf{x}) - y)^2] = E_{(\mathbf{x}, y), D} [(h_D(\mathbf{x}) - \bar{h}(\mathbf{x}))^2] + E_{(\mathbf{x}, y), D} [(\bar{h}(\mathbf{x}) - y)^2]$$

We also have

$$\begin{aligned} E_{(\mathbf{x},y),D}[(\bar{h}(\mathbf{x}) - y)^2] &= E_{(\mathbf{x},y),D}[(\bar{h}(\mathbf{x}) - t(\mathbf{x})) + (t(\mathbf{x}) - y)]^2 \\ &= E_{(\mathbf{x},y)}[(t(\mathbf{x}) - y)^2] + E_{(\mathbf{x},y)}[(\bar{h}(\mathbf{x}) - t(\mathbf{x}))^2] + 2E_{(\mathbf{x},y)}[(t(\mathbf{x}) - y)(\bar{h}(\mathbf{x}) - t(\mathbf{x}))] \\ &= E_{(\mathbf{x},y)}[(t(\mathbf{x}) - y)^2] + E_{(\mathbf{x},y)}[(\bar{h}(\mathbf{x}) - t(\mathbf{x}))^2], \end{aligned}$$

where we utilize

$$\begin{aligned} E_{(\mathbf{x},y)}[(t(\mathbf{x}) - y)(\bar{h}(\mathbf{x}) - t(\mathbf{x}))] &= E_{\mathbf{x}}\left[E_{y|\mathbf{x}}[(t(\mathbf{x}) - y)(\bar{h}(\mathbf{x}) - t(\mathbf{x}))]\right] \\ &= E_{\mathbf{x}}\left[(t(\mathbf{x}) - E_{y|\mathbf{x}}(y))(\bar{h}(\mathbf{x}) - t(\mathbf{x}))\right] \\ &= E_{\mathbf{x}}\left[(t(\mathbf{x}) - t(\mathbf{x}))(\bar{h}(\mathbf{x}) - t(\mathbf{x}))\right] = 0 \end{aligned}$$

Finally, we have

$$\begin{aligned} E_{(\mathbf{x},y),D}[(h_D(\mathbf{x}) - y)^2] \\ = E_{(\mathbf{x},y),D}[(h_D(\mathbf{x}) - \bar{h}(\mathbf{x}))^2] + E_{(\mathbf{x},y)}[(\bar{h}(\mathbf{x}) - t(\mathbf{x}))^2] + E_{(\mathbf{x},y)}[(t(\mathbf{x}) - y)^2] \end{aligned}$$

Above three terms are **variance**, **bias²**, **noise**, respectively.

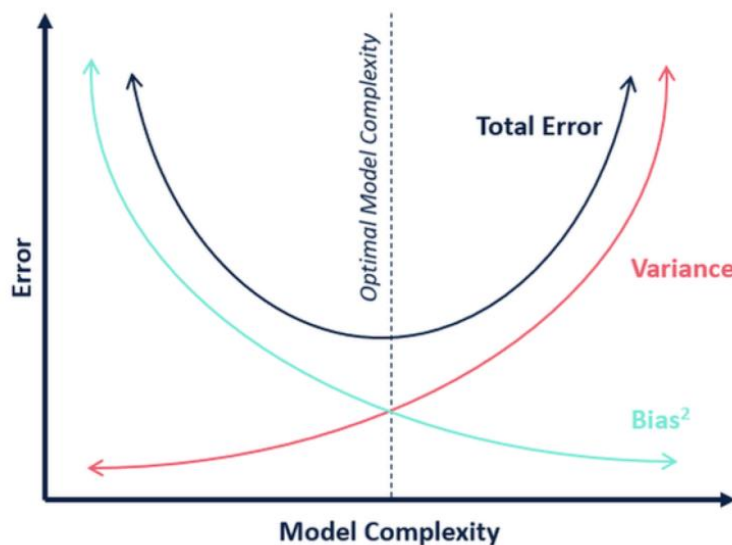
(这都几把什么玩意?)

但是总之呢，总误差=偏差²+方差+噪音

方差与训练集内容有关，显示了对训练集的过拟合程度(方差越高过拟合越严重)

偏差与模型有关，显示了模型的复杂度(偏差越低复杂度越高)

噪音和数据本身的分布、特征有关，无法解决



重申：过拟合可以用降低模型复杂度，正则化，集成，增加训练数据等方法解决

欠拟合可以通过提高模型复杂度，增加特征数量来解决

实例化计算看一眼就知道了：

Exercise:

- For a new test sample $(x, y) = (5, 10)$ sampled from the same distribution that generated the training sets, we suppose $t(x = 5) = 9.5$, and ϵ is instantiated as 0.5.
- Suppose the predictions of this new test sample based on the 10 trained models are 9, 11, 23, 6, 8, 12, 10, 4, 13, 7, respectively.
- Based on this 10 trials, please compute the **empirical mean squared error (MSE)**, **Bias²** and **Variance** on this test sample.
- Average Prediction = $(9 + 11 + 23 + 6 + 8 + 12 + 10 + 4 + 13 + 7)/10 = 10.3$
Bias² = $(10.3 - 9.5)^2 = 0.64$
Variance = $\frac{1}{10}[(9 - 10.3)^2 + (11 - 10.3)^2 + (23 - 10.3)^2 + (6 - 10.3)^2 + (8 - 10.3)^2 + (12 - 10.3)^2 + (10 - 10.3)^2 + (4 - 10.3)^2 + (13 - 10.3)^2 + (7 - 10.3)^2]$
= 24.81
Empirical MSE = $\frac{1}{10}[(9 - 10)^2 + (11 - 10)^2 + (23 - 10)^2 + (6 - 10)^2 + (8 - 10)^2 + (12 - 10)^2 + (10 - 10)^2 + (4 - 10)^2 + (13 - 10)^2 + (7 - 10)^2] = 24.9$.

看得出来，MSE 是（每个预测值-原始标签）²取平均（也就是 $h(x) - 10$ ）

偏差是预测值的平均值-排除了噪音的真实标签（也就是 $h(x) - 9.5$ ）

方差是（每个预测值-预测平均值）²

Performance Evaluation

1. Cross Validation

用于解决超参数(learning rate, power of polynomial, C in SVM, depth in DT)

K 折交叉检验的原理也很简单，一张图通俗易懂

fold 1	fold 2	fold 3	fold 4	fold 5	test
fold 1	fold 2	fold 3	fold 4	fold 5	test
fold 1	fold 2	fold 3	fold 4	fold 5	test

K 过大，验证集的样本数量就少，导致过拟合；k 过小，我们很可能找不到有意义的答案，导致欠拟合。一般而言， $5 \leq k \leq 10$

2. Evaluation metrics (以下全文背诵)

$$\text{MSE} = \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n}. \quad \text{MAE} = \frac{\sum_{i=1}^n |y_i - \hat{y}_i|}{n}$$

A **confusion matrix** consists of four entries:

- **True Positive (TP)**: a positive sample is classified (predicted) as positive sample.
- **True Negative (TN)**: a negative sample is classified (predicted) as negative sample.
- **False Negative (FN)**: a positive sample is classified (predicted) as negative sample.
- **False Positive (FP)**: a negative sample is classified (predicted) as positive sample.

	$\hat{P}$ (predicted)	$\hat{N}$ (predicted)	
P (actual)	TP	FN	Recall $TP/(TP+FN)$
N (actual)	FP	TN	
	Precision $TP/(TP+FP)$	Accuracy $(TP+TN)/(TP+TN+FP+FN)$	

注：背诵小技巧：F/T 表示预测正确与否，P/N 表示模型预测的结果

准确率=预测对的/总数 $\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$

查准率=预测对的正例/预测正例 $\text{Precision} = \frac{TP}{TP + FP}$

召回率：预测对的正例/真正例 $\text{Recall} = \frac{TP}{TP + FN}$

(True Positive Rate) $\text{TPR} = TP/(TP+FN)$

(False Negative Rate) $\text{FNR} = FN/(TP+FN)$

(True Negative Rate) $\text{TNR} = TN/(FP+TN)$

(False Positive Rate) $\text{FPR} = FP/(FP+TN)$

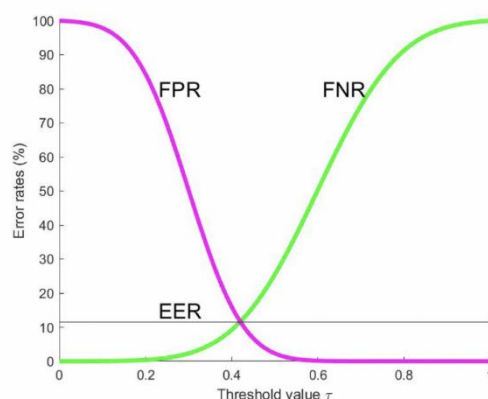
注：背诵小技巧：TP 和 FN 是一对，TN 和 FP 是一对，想想碱基互补配对

If positive and negative classes are balanced, then the accuracy could be computed by

$$\text{Accuracy} = \frac{TPR + TNR}{2} = 1 - \frac{FPR + FNR}{2}$$

该等式当且仅当正负例个数完全相同时成立

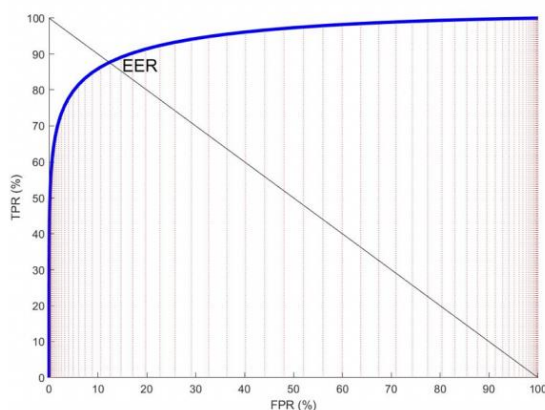
The intersection point of these two curves is when the two error rates are equal (i.e., $FPR=FNR$). This point is called the [Equal Error Rate \(EER\)](#).



Threshold value 的意思是阈值，大于该值判定为正，否则为负

The [Area Under the ROC \(AUC\)](#) can also be used to gauge the classifier's accuracy (in overall sense, i.e., under all operating points).

ROC 曲线：



AUC 的值不会随阈值的变化而变化，因为它本身就包含了所有阈值

手算 AUC: $e_{ij} = g(\mathbf{x}_i^+) - g(\mathbf{x}_j^-)$

Area Under the ROC Curve (AUC)

- Consider also a Heaviside step function given by

$$u(e) = \begin{cases} 1, & \text{if } e > 0 \\ 0.5 & \text{if } e = 0 \\ 0 & \text{if } e < 0 \end{cases}$$

- The Area Under the ROC Curve (AUC) can be expressed as

$$\text{AUC} = \frac{1}{m^+m^-} \sum_{i=1}^{m^+} \sum_{j=1}^{m^-} u(e_{ij})$$

Unsupervised learning:

简要讲讲无监督学习的作用

1. Clustering, 聚类, 如 Kmeans
2. Dimension reduction, 降维, 如 PCA
3. Density estimation, 分布估计, 如 KDE
4. Autoencoder, 自动编码器, 用于恢复图片, 制作视频等
5. Self-supervised learning, 半监督学习

Kmeans

Kmean 的过程:

1. 选择 K, 并初始化簇中心
2. 将样本分配给离它最近的簇中心
3. 将簇中心移动至分配给它的所有样本的中心
4. 重复 2&3 直到收敛

我们实际上是在优化这个函数，翻译一下，这个函数的含义是所有簇的方差的和

$$\min_{\mathbf{c}, \mathbf{r}} J(\mathbf{c}, \mathbf{r}) = \min_{\mathbf{c}, \mathbf{r}} \sum_i^n \sum_k^K r_{ik} (\mathbf{x}_i - \mathbf{c}_k)^2,$$

$$\text{Subject to } \mathbf{r} \in \{0, 1\}^{n \times K}, \sum_k^K r_{ik} = 1,$$

where $r_{ik} = 1$ denotes $\mathbf{x}_i$ is assigned to cluster k .

固定 $\mathbf{c}$ (簇) 的过程叫 Assignment, 固定 $\mathbf{r}$ (标签) 的过程叫 Refitting

Note that the assignment for each data $\mathbf{x}_i$ can be solved independently *i.e.*,

$$\min_{\mathbf{r}_i} \sum_k^K r_{ik} (\mathbf{x}_i - \mathbf{c}_k)^2, \text{ subject to } \mathbf{r}_i \in \{0, 1\}^{1 \times K}, \sum_k^K r_{ik} = 1.$$

注意到 $\mathbf{x}_i$ 之间实际上是独立的，所以把他们分别最小化即可，答案一眼就能看出来，即把 $\mathbf{x}_i$ 分给距离最近的 $\mathbf{c}_k$

同理，Refitting 的过程也相似

Note that $\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_K$ can be optimized independently, as follows

$$\min_{\mathbf{c}_k} \sum_i^n r_{ik} (\mathbf{x}_i - \mathbf{c}_k)^2.$$

By setting the derivative *w.r.t.* $\mathbf{c}_k$ as 0, *i.e.*,

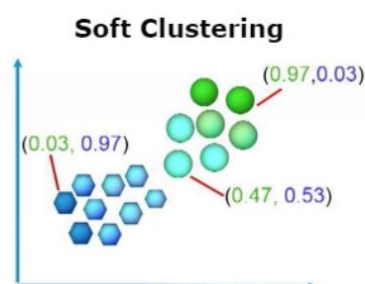
$$\sum_i^n 2r_{ik} (\mathbf{x}_i - \mathbf{c}_k) = 0 \Rightarrow \mathbf{c}_k = \frac{\sum_i^n r_{ik} \mathbf{x}_i}{\sum_i^n r_{ik}},$$

这也就是让 $\mathbf{c}_k$ = 簇内平均值

由于这个优化函数不是 convex 的，Kmeans 有可能会陷入局部最优。解决方案

是用不同的初始化簇中心多跑几次

Soft clustering:



我们的优化函数改为：

$$\min_{\mathbf{c}, \mathbf{r}} J(\mathbf{c}, \mathbf{r}) = \min_{\mathbf{c}, \mathbf{r}} \sum_i^n \sum_k^K (r_{ik})^m (\mathbf{x}_i - \mathbf{c}_k)^2,$$

$$\text{Subject to } \mathbf{r} \in [0, 1]^{n \times K}, \sum_k^K r_{ik} = 1,$$

M 也是超参数，被称作 fuzzifier，越接近 1，该优化函数越接近 hard clustering

这里我们用拉格朗日函数和 KKT 条件求解

$$\mathcal{L}(\mathbf{r}, \boldsymbol{\alpha}, \boldsymbol{\beta}) = J(\mathbf{r}) + \sum_i^n \alpha_i \left(1 - \sum_k^K r_{ik}\right) + \sum_i^n \sum_k^K \beta_{ik} (-r_{ik})$$

这里没有把 $r \leq 1$ 这一条件也纳入拉格朗日函数，因为 $\sum_k^K r_{ik} = 1$ 就蕴含了这个条件

The KKT conditions are:

$$\text{Stationary: } \frac{\partial \mathcal{L}}{\partial r_{ik}} = 0 \Rightarrow r_{ik} = \left(\frac{\alpha_i + \beta_{ik}}{m}\right)^{\frac{1}{m-1}} \cdot \left(\frac{1}{d_{ik}^2}\right)^{\frac{1}{m-1}}$$

$$\text{Primal feasibility: } \sum_k^K r_{ik} = 1, r_{ik} \geq 0$$

$$\text{Dual feasibility: } \beta_{ik} \geq 0, \forall i, k$$

$$\text{Complementary: } \beta_{ik} \cdot r_{ik} = 0, \forall i, k,$$

where $d_{ik}^2 = (\mathbf{x}_i - \mathbf{c}_k)^2$.

解得：

$$r_{ik} = \frac{1}{\sum_j^K \left(\frac{1}{d_{ij}^2}\right)^{\frac{1}{m-1}}} \cdot \left(\frac{1}{d_{ik}^2}\right)^{\frac{1}{m-1}} = \frac{1}{\sum_j^K \left(\frac{d_{ik}^2}{d_{ij}^2}\right)^{\frac{1}{m-1}}}.$$

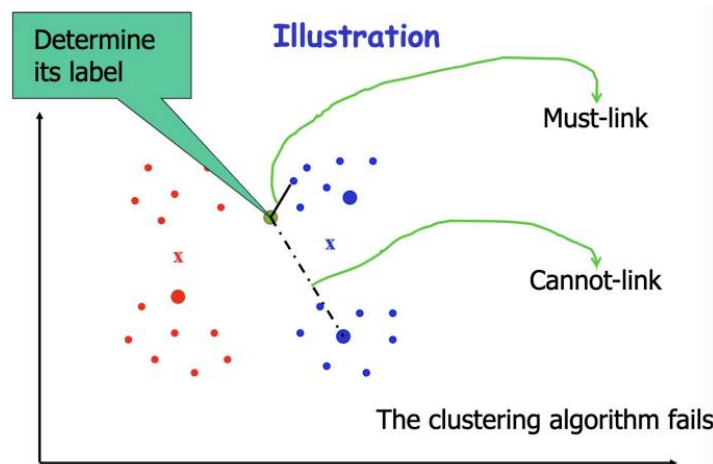
对 $\mathbf{c}$ 求导，解得

$$\frac{\partial J(\mathbf{c})}{\partial \mathbf{c}_k} = 0 \Rightarrow \mathbf{c}_k = \frac{\sum_i^n [(r_{ik})^m \mathbf{x}_i]}{\sum_i^n (r_{ik})^m}.$$

对于有限制的 Kmeans：

必连/勿连限制：在给样本点分配簇时，检查限制条件，如冲突则分配至第二近的簇中心

注意，此种情况下，Kmeans 会 fail



时间复杂度：Assignment 中，计算距离所需的时间是 $O(d)$ ，一共有 K 个簇，所以要计算 K 次，一共有 n 个样本，所以要重复 n 次该过程，共 $O(n \cdot K \cdot d)$

Refitting 中，计算每个簇的平均值需要 $O(d \cdot n_k)$ 的复杂度， n_k 代表该簇的样本数。由于所有 n_k 相加为 n ，所以时间复杂度为 $O(d \cdot n)$

总复杂度为： $O(T \cdot n \cdot (K+1) \cdot d)$ ， T 是循环次数

评价标准：Silhouette coefficient，用于评价某个 Kmeans 的好坏，越大越好

Given a clustering, we define

- a : The mean distance between a point and all other points in the **same** cluster.
- b : The mean distance between a point and all other points in the next **nearest** cluster.

Silhouette coefficient s for a single sample is formulated as:

$$s = \frac{b - a}{\max(a, b)} \Rightarrow s = \begin{cases} 1 - \frac{a}{b} & \text{if } a < b \\ 0 & \text{if } a = b \\ \frac{b}{a} - 1 & \text{if } a > b \end{cases}$$

Rand Index, 用于评价两个 Kmeans 模型的相似程度，越高越像

Given a set of n samples $S = \{o_1, o_2, \dots, o_n\}$, there are two clusterings/partitions of S to compare, including:

- $X = \{X_1, X_2, \dots, X_r\}$ with r clusters
- $Y = \{Y_1, Y_2, \dots, Y_s\}$ with s clusters

We can calculate the following values:

- a : The number of pairs of elements in S that are in the **same** subset in X and in the **same** subset in Y
- b : The number of pairs of elements in S that are in the **different** subset in X and in the **different** subset in Y
- c : The number of pairs of elements in S that are in the **same** subset in X and in the **different** subset in Y
- d : The number of pairs of elements in S that are in the **different** subset in X and in the **same** subset in Y

The **rand index** (RI) can be computed as follows:

$$RI = \frac{a + b}{a + b + c + d} = \frac{a + b}{\frac{n(n-1)}{2}}$$

GMM and EM algorithm (最抽象的一集)

However, we do not have the class labels z in unsupervised clustering. In this case, we can model the marginal distribution over $\mathbf{x}$ as follows

$$p(\mathbf{x}) = \sum_z p(\mathbf{x}, z) = \sum_z p(\mathbf{x}|z)p(z)$$

This is called **mixture models**.

也就是说, 由混合分布产生 $\mathbf{x}$ 的概率是混合分布中每个分布产生 $\mathbf{x}$ 的概率的总和

A GMM represents a **distribution** as

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

where π_k are the **mixing coefficients**, $\sum_{k=1}^K \pi_k = 1$, $\pi_k \geq 0, \forall k$, and

$$\mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = \frac{1}{\sqrt{(2\pi)^d |\boldsymbol{\Sigma}_k|}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1}(\mathbf{x} - \boldsymbol{\mu}_k)\right),$$

with $|\boldsymbol{\Sigma}_k| = \det(\boldsymbol{\Sigma}_k)$ denotes the determinant of $\boldsymbol{\Sigma}_k$, d indicates the dimension of $\mathbf{x}$.

GMM is a **density estimator**. If given enough Gaussian components, GMM is **universal approximator of densities**.

(放一页 PPT 在这，假装我看懂了，尽管我啥也没理解)

但至少可以知道的是，GMM 是用来估计 $\mathbf{x}$ 的分布的

The log-likelihood is

$$\log \mathcal{L}(\Theta) = \ln p(\mathbf{X} | \pi, \mu, \Sigma) = \sum_{n=1}^N \ln \left(\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}^{(n)} | \mu_k, \Sigma_k) \right)$$

where $\mathbf{X} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}\}$, $\Theta = \{\pi, \mu, \Sigma\}$, $\pi = \{\pi_1, \dots, \pi_K\}$, $\mu = \{\mu_1, \dots, \mu_K\}$, and $\Sigma = \{\Sigma_1, \dots, \Sigma_K\}$.

We aim to learn the parameters Θ by maximizing the above log-likelihood.

Let the derivative of $\mathcal{L}$ w.r.t. μ_k be zero:

$$\frac{\partial \mathcal{L}}{\partial \mu_k} = 0 \implies \sum_{n=1}^N \underbrace{\frac{\pi_k \mathcal{N}(\mathbf{x}^{(n)} | \mu_k, \Sigma_k)}{\sum_j \pi_j \mathcal{N}(\mathbf{x}^{(n)} | \mu_j, \Sigma_j)}}_{\gamma_k^{(n)}} \Sigma_k^{-1} (\mathbf{x}^{(n)} - \mu_k) = 0$$
$$\mu_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_k^{(n)} \mathbf{x}^{(n)}, \text{ where } N_k = \sum_{n=1}^N \gamma_k^{(n)}$$

求导确实是这个结果，虽然我既不知道为什么要求导，也不知道接下来要干嘛

$$\frac{\partial \mathcal{L}}{\partial \Sigma_k} = 0 \implies \Sigma_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_k^{(n)} (\mathbf{x}^{(n)} - \mu_k)(\mathbf{x}^{(n)} - \mu_k)^\top$$

: $\tilde{\mathcal{L}} := \mathcal{L} + \lambda(\sum_{k=1}^K \pi_k - 1)$, λ : Lagrange multiplier

$$\frac{\partial \tilde{\mathcal{L}}}{\partial \pi_k} = 0 \implies \sum_{n=1}^N \frac{\mathcal{N}(\mathbf{x}^{(n)} | \mu_k, \Sigma_k)}{\sum_j \pi_j \mathcal{N}(\mathbf{x}^{(n)} | \mu_j, \Sigma_j)} + \lambda = 0$$

因为权重的总和是 1，所以用拉格朗日进行限制

Initialize μ_k, Σ_k , and $\pi_k, k = 1, \dots, K$. Iterate until convergence:

- **Step 1:** Evaluate the responsibilities given current parameters

$$\gamma_k^{(n)} = \frac{\pi_k \mathcal{N}(\mathbf{x}^{(n)} | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}^{(n)} | \mu_j, \Sigma_j)}, \quad k = 1, \dots, K, \quad n = 1, \dots, N.$$

- **Step 2:** Re-estimate the parameters given current responsibilities

$$\mu_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_k^{(n)} \mathbf{x}^{(n)},$$

$$\Sigma_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_k^{(n)} (\mathbf{x}^{(n)} - \mu_k) (\mathbf{x}^{(n)} - \mu_k)^\top,$$

$$\pi_k = \frac{N_k}{N}, \text{ with } N_k = \sum_{n=1}^N \gamma_k^{(n)}.$$

- Evaluate the log-likelihood and check for convergence

$$\ln p(\mathbf{X} | \pi, \mu, \Sigma) = \sum_{n=1}^N \ln \left(\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}^{(n)} | \mu_k, \Sigma_k) \right).$$

This algorithm is actually the well-known Expectation-Maximization (EM) algorithm, where Steps 1 and 2 are called **E-step** and **M-step** respectively.

现在知道他想干嘛了。先定 theta 求 y，再定 y 求 theta。前者叫做 E 步，后者叫做 M 步

当协方差矩阵=I 时，GMM 退化为 soft Kmeans。从聚类效果上看，GMM 和 Kmeans 的最终效果差不多（或者说 GMM 是 Kmeans 的上位替代）

同样，GMM 的 K 和初始值对结果影响很大，且无法处理高维度数据

PCA

Preliminary: Projection onto a subspace

基础

线代

知识

- Given a dataset $\mathcal{D} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}\}$ with $\mathbf{x}^{(n)} \in \mathbb{R}^D$ and D being the original dimension. And we define the mean as $\mu = \frac{1}{N} \sum_{n=1}^N \mathbf{x}^{(n)} \in \mathbb{R}^D$.
- K -dimensional subspace $\mathcal{S}$ is spanned by an orthonormal basis $\{\mathbf{u}_k\}_{k=1}^K$ with $\mathbf{u}_k \in \mathbb{R}^D$.
- $\|\mathbf{u}_k\| = 1, \forall k; \mathbf{u}_i^\top \mathbf{u}_j = 0$ if $i \neq j, \forall i, j$.
- Approximate each data point $\mathbf{x} \in \mathbb{R}^D$ as:

$$\tilde{\mathbf{x}} = \mu + \text{Proj}_{\mathcal{S}}(\mathbf{x} - \mu) = \mu + \sum_{k=1}^K z_k \mathbf{u}_k,$$

where $z_k = \mathbf{u}_k^\top (\mathbf{x} - \mu)$ can be seen as the projection length of $\mathbf{x} - \mu$ on the k -th basis $\mathbf{u}_k$.

- Let $\mathbf{U} \in \mathbb{R}^{D \times K}$ be a matrix with columns $\{\mathbf{u}_k\}_{k=1}^K$, then we have

$$\tilde{\mathbf{x}} = \mu + \mathbf{U} \mathbf{z} \in \mathbb{R}^D, \text{ which is called reconstruction of } \mathbf{x} \quad (1)$$

$$\mathbf{z} = \mathbf{U}^\top (\mathbf{x} - \mu) \in \mathbb{R}^K, \text{ which is called representation/code of } \mathbf{x}. \quad (2)$$

注意 $\tilde{\mathbf{x}}$ 叫做 $\mathbf{x}$ 的重构, $\mathbf{z}$ 叫做 $\mathbf{x}$ 的投影

Theorem (Orthogonal theorem)

The vector $\mathbf{x} - \tilde{\mathbf{x}}$ is *orthogonal* to the subspace $\mathcal{S}$, i.e.,

$$\mathbf{U}^\top (\mathbf{x} - \tilde{\mathbf{x}}) = \mathbf{0}.$$

很好理解, 毕竟如果不垂直的话, 就不叫投影了

PCA 就是这样一种降维手段, 输入 d 维的 $\mathbf{x}$, 输出 k 维的 $\mathbf{z}$

Given a dataset $\mathcal{D} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}\} \in \mathbb{R}^D$, we want to find a K -dimensional ($K < D$) subspace $\mathcal{S}$, which consists of K orthonormal basis vectors $\{\mathbf{u}_k\}_{k=1}^K$, such that the variance of the reconstructions $\tilde{\mathcal{D}} = \{\tilde{\mathbf{x}}^{(1)}, \dots, \tilde{\mathbf{x}}^{(N)}\}$ is maximal, i.e.,

$$\max_{\mathbf{U}, \mathbf{U}^\top \mathbf{U} = \mathbf{I}} \frac{1}{N} \sum_{n=1}^N \|\tilde{\mathbf{x}}^{(n)} - \tilde{\boldsymbol{\mu}}\|^2, \quad (3)$$

为什么要最大化重构方差呢? 因为方差代表着其蕴含的信息量。我们应该尽可能的保留数据点之间的差异性, 并尽可能去除数据点之间的相似性

这里的证明都很简单, 把 $\mathbf{x}$ 和 $\mathbf{z}$ 代入一下就可得到:

$$\begin{aligned} & \max_{\mathbf{U}, \mathbf{U}^\top \mathbf{U} = \mathbf{I}} \frac{1}{N} \sum_{n=1}^N \|\mathbf{U}^\top (\mathbf{x} - \boldsymbol{\mu})\|^2 \\ & \equiv \max_{\mathbf{U}, \mathbf{U}^\top \mathbf{U} = \mathbf{I}} \frac{1}{N} \sum_{n=1}^N \text{Trace}(\mathbf{U}^\top (\mathbf{x}^{(n)} - \boldsymbol{\mu})(\mathbf{x}^{(n)} - \boldsymbol{\mu})^\top \mathbf{U}). \end{aligned}$$

另一个角度: 最小化重构损失

这个也很好理解, 我们要去除最不重要的特征, 即去除后损失最小的特征

Given a dataset $\mathcal{D} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}\} \in \mathbb{R}^D$, we want to find a K -dimensional ($K < D$) subspace $\mathcal{S}$, which consists of K orthonormal basis vectors $\{\mathbf{u}_k\}_{k=1}^K$, such that the reconstruction loss between $\mathbf{x}$ and $\tilde{\mathbf{x}}$ is minimized, i.e.,

$$\min_{\mathbf{U}, \mathbf{U}^\top \mathbf{U} = \mathbf{I}} \frac{1}{N} \sum_{n=1}^N \|\mathbf{x}^{(n)} - \tilde{\mathbf{x}}^{(n)}\|^2. \quad (9)$$

这两个优化问题是等价的, 同样可以通过简单的代入得证 (也是作业三的手写题)

Until now, we have known that PCA aims to solve the following optimization problem,

$$\max_{\mathbf{U}, \mathbf{U}^\top \mathbf{U} = \mathbf{I}} \frac{1}{N} \sum_{n=1}^N \text{Trace}(\mathbf{U}^\top (\mathbf{x}^{(n)} - \boldsymbol{\mu})(\mathbf{x}^{(n)} - \boldsymbol{\mu})^\top \mathbf{U}). \quad (11)$$

We define the empirical covariance matrix, as follows:

$$\boldsymbol{\Sigma} = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}^{(n)} - \boldsymbol{\mu})(\mathbf{x}^{(n)} - \boldsymbol{\mu})^\top. \quad (12)$$

Then, the above optimization can be reformulated as follows:

$$\max_{\mathbf{U}} \text{Trace}(\mathbf{U}^\top \boldsymbol{\Sigma} \mathbf{U}) = \sum_{k=1}^K \mathbf{u}_k^\top \boldsymbol{\Sigma} \mathbf{u}_k, \text{ s.t. } \mathbf{U}^\top \mathbf{U} = \mathbf{I}. \quad (13)$$

The Lagrangian function is formulated as follows:

$$L(\mathbf{U}, \boldsymbol{\Lambda}_K) = \text{Trace}(\mathbf{U}^\top \boldsymbol{\Sigma} \mathbf{U}) + \text{Trace}(\boldsymbol{\Lambda}_K^\top (\mathbf{I} - \mathbf{U}^\top \mathbf{U}))$$

where $\boldsymbol{\Lambda}_K = \text{diag}([\hat{\lambda}_1, \dots, \hat{\lambda}_K]) \in \mathbb{R}^{K \times K}$.

这大概是拉格朗日方程的推广版本，讲是不讲的，会是要会的，nmsl

$$\begin{aligned} \frac{\partial L(\mathbf{U}, \boldsymbol{\Lambda}_K)}{\partial \mathbf{U}} &= 2\boldsymbol{\Sigma} \mathbf{U} - 2\mathbf{U} \boldsymbol{\Lambda}_K = \mathbf{0} \\ \Rightarrow \boldsymbol{\Sigma} \mathbf{u}_k &= \hat{\lambda}_k \mathbf{u}_k, \quad k = 1, \dots, K. \end{aligned}$$

Utilizing SVD decomposition, we have

$$\boldsymbol{\Sigma} = \mathbf{Q} \boldsymbol{\Lambda}_D \mathbf{Q}^\top = \sum_{i=1}^D \lambda_i \mathbf{q}_i \mathbf{q}_i^\top,$$

where $\mathbf{Q} = [\mathbf{q}_1, \dots, \mathbf{q}_D] \in \mathbb{R}^{D \times D}$ with $\mathbf{q}_i$ being the eigenvector corresponding to the i -th largest eigenvalue λ_i , and $\boldsymbol{\Lambda}_D = \text{diag}([\lambda_1, \dots, \lambda_D])$ with $\lambda_1 \geq \dots \geq \lambda_D$.

$$\sum_{k=1}^K \mathbf{u}_k^\top \boldsymbol{\Sigma} \mathbf{u}_k = \sum_{k=1}^K \sum_{i=1}^D \lambda_i (\mathbf{u}_k^\top \mathbf{q}_i) \cdot (\mathbf{q}_i^\top \mathbf{u}_k) = \sum_{t \in T} \lambda_t$$

咱也不知道这式子是怎么得出来的，反正结论就是把协方差矩阵的特征值从大到

小排序，对应的前 k 列特征向量就是投影矩阵